Aress Frontend Project - Comprehensive Technical Documentation

Table of Contents

Project Overview

The **Aress Frontend Project** is a sophisticated enterprise-grade monorepo built with Nx, designed to support multiple frontend applications with a shared design system. The project emphasizes code reusability, maintainability, and developer experience through modern tooling and best practices.

Key Features

- **Monorepo Architecture**: Nx-powered workspace for efficient code sharing
- **Comprehensive Design System**: 60+ reusable UI components
- **Multiple Applications**: B2B and B2C applications with shared infrastructure

- **Modern Tooling**: TypeScript, Tailwind CSS, Storybook, Vitest
- **API-First Approach**: OpenAPI/Swagger integration with code generation
- **Production Ready**: Docker support, CI/CD workflows, and quality gates

---

Architecture & Technology Stack

Core Technologies

- **Framework**: React 18+ with Next.js 15
- **Language**: TypeScript 5+
- **Styling**: Tailwind CSS with custom design tokens
- **Build System**: Nx 21.3.0 with Vite/Webpack
- **Package Manager**: pnpm (workspace support)

Development Tools

- **Component Development**: Storybook 9.0
- **Testing**: Vitest + Jest + Playwright
- **Code Quality**: ESLint, Prettier, Husky, Commitlint
- **Icons**: Iconify with Lucide icon set
- **Documentation**: Storybook with auto-docs

Infrastructure

- **Containerization**: Docker with multi-stage builds
- **Deployment**: Vercel integration
- **API Integration**: OpenAPI code generation
- **State Management**: React Query (TanStack Query)

Project Structure

```
aress-frontend/
├── apps/                          # Applications
│   ├── fe-app/                   # Main B2B application (Next.js)
│   ├── b2c-app/                  # B2C application (Next.js)
│   ├── fe-app-e2e/              # E2E tests for fe-app
```

```
│   └── renderer/                # Rendering service
├── libs/                         # Shared libraries
│   ├── design-system/           # UI component library
│   ├── openapi/                 # Generated API clients
│   └── shared/                  # Shared utilities and hooks
├── scripts/                      # Build and utility scripts
├── shared/                       # Global shared resources
├── .storybook/                  # Global Storybook configuration
├── docker-compose.yaml          # Development environment
├── nx.json                       # Nx workspace configuration
└── package.json                 # Root package configuration
```

Design System - Deep Dive

Overview

The design system is located in `libs/design-system` and provides a comprehensive set of 60+ reusable UI components built with React, TypeScript, and Tailwind CSS.

Component Architecture

Each component follows a consistent structure:

```
ComponentName/
├── ComponentName.tsx              # Main component implementation
├── ComponentName.types.ts         # TypeScript interfaces (if
multiple)
├── ComponentName.constants.ts     # Component constants (if needed)
├── ComponentName.stories.tsx      # Storybook stories
├── ComponentName.test.tsx         # Unit tests
└── index.ts                       # Exports
```

Complex Components Analysis

1. VideoPlayer Component - Advanced Implementation

Location: `libs/design-system/src/lib/components/VideoPlayer/`

The VideoPlayer is one of the most sophisticated components in the design system, featuring a complete custom video player implementation with advanced controls and state management.

Architecture Overview

The VideoPlayer consists of multiple interconnected parts:

1. **Main VideoPlayer Component** (`VideoPlayer.tsx`)
2. **Custom Hook** (`UseVideo.tsx`) - State management with useReducer
3. **Control Panel Subcomponents**:
   - `PlayerActions` - Play/pause, volume controls
   - `PlayerOptions` - Quality, fullscreen, PiP controls
   - `PlayerProgressBar` - Seek bar with thumbnail preview
   - `PlayerThumbnail` - Hover thumbnails on progress bar
   - `VideoTimer` - Current time and duration display

State Management Implementation

The VideoPlayer uses a sophisticated state management system with `useReducer`:

```
interface videoState {
  isPlaying: boolean;
  currentTime: number;
  duration: number;
  isFinished: boolean;
  progress: number;
  isVideoLoaded: boolean;
  isVideoWaited: boolean; // buffering state
  bufferedTime: number;
  playBackRate: number;
  volume: number;
  muted: boolean;
  isFullscreen: boolean;
  quality: { src: string; label: string; };
  error: string | null;
  src: string;
}
```

Key State Actions: - `PLAY/PAUSE` - Control playback state - `TIME_UPDATE` - Track current playback time - `SET_VIDEO_LOADED/WAITED` - Handle loading and buffering states - `SET_QUALITY` - Dynamic quality switching - `SET_FULLSCREEN` - Fullscreen mode management

Advanced Features

1. HLS.js Integration

```
import Hls from 'hls.js';
// Supports adaptive streaming for different video qualities
```

2. Control Panel Auto-Hide

```
useEffect(() => {
  let timeout: NodeJS.Timeout;
  const showControls = () => {
    setShowControlPanel(true);
    clearTimeout(timeout);
    timeout = setTimeout(() => setShowControlPanel(false), 5000);
  };
  // Auto-hide controls after 5 seconds of inactivity
}, []);
```

3. Playlist Integration - Fullscreen playlist overlay - Video switching capabilities - Playlist state management - Click-outside-to-close functionality

4. Thumbnail Preview System

```
spriteBaseUrl?: {
  image: string;
  intervalSeconds: number;
};
```

- Hover thumbnails on progress bar
- Sprite-based thumbnail system for performance
- Configurable interval timing

5. Keyboard Navigation Support - Space bar for play/pause - Arrow keys for seeking - Volume controls - Fullscreen toggle

6. Picture-in-Picture Support - Native browser PiP API integration - Fallback handling for unsupported browsers

Performance Optimizations

1. Memoized Components

```
const MemoizedTitle = React.memo(({ title, setShowPlayList }) => {
  // Prevents unnecessary re-renders of title component
});
```

2. Event Listener Cleanup

```
useEffect(() => {
  return () => {
    videoContainerRef.current?.removeEventListener('mousemove',
        showControls);
    clearTimeout(timeout);
  };
}, []);
```

3. Conditional Rendering - Controls only render when needed - Playlist overlay only in fullscreen mode - Loading states prevent interaction until ready

Usage Example

```
<VideoPlayer
  qualities={[
    { src: 'video-720p.mp4', label: '720p' },
    { src: 'video-1080p.mp4', label: '1080p' }
  ]}
  poster="thumbnail.jpg"
  title="Investment Fund Analysis"
  src="video.mp4"
  spriteBaseUrl={{
    image: 'thumbnails-sprite.jpg',
    intervalSeconds: 10
  }}
  videos={playlistVideos}
  selectedVideo={currentVideo}
  setSelectedVideo={setCurrentVideo}
/>
```

2. GeneralTable Component - Data Management

Location: `libs/design-system/src/lib/components/GeneralTable/`

The GeneralTable is a highly sophisticated data table component built on top of TanStack Table (React Table v8) with advanced features for enterprise data management.

Key Features

1. Column Management - Drag-and-drop column reordering - Column visibility controls - Column resizing - Sticky columns (pinning) - Custom column types

2. Sorting & Filtering - Multi-column sorting - Advanced filtering with multiple operators - Date range filtering - Custom filter components - Filter persistence

3. Data Export - Excel export functionality - PDF export capabilities - Custom export formats - Filtered data export

4. Performance Features - Virtual scrolling for large datasets - Pagination with configurable page sizes - Lazy loading support - Optimized re-rendering

5. User Experience - Row selection (single/multiple) - Row highlighting and marking - Keyboard navigation - Responsive design - Loading states and skeletons

Implementation Details

Column Definition System:

```
interface FundColumnMeta {
  dragIndicator?: boolean;
  isSticky?: boolean;
  stickyPosition?: number;
  filterType?: 'text' | 'number' | 'date' | 'select';
  exportable?: boolean;
}
```

Advanced Sorting: - Server-side sorting integration - Multi-column sort with priority - Custom sort functions - Sort state persistence

Filter System: - Multiple filter types (text, number, date, select) - Range filters for numerical data - Date picker integration - Custom filter components

3. Form Components - Validation & UX

TextField Component

Advanced Features: - Real-time validation - Debounced input handling - Custom validation rules - Error state management - Accessibility compliance (ARIA labels)

```
interface TextFieldProps {
  validation?: {
    required?: boolean;
    minLength?: number;
    maxLength?: number;
    pattern?: RegExp;
    custom?: (value: string) => string | null;
  };
  debounceMs?: number;
  onValidationChange?: (isValid: boolean) => void;
}
```

Checkbox & Radio Components

Features: - Group management - Indeterminate state support - Custom styling with CSS-in-JS - Keyboard navigation - Form integration

4. Dialog/Modal System

Advanced Modal Management: - Modal stacking support - Focus trap implementation - Escape key handling - Backdrop click handling - Animation system - Portal rendering

```
interface DialogProps {
  isOpen: boolean;
  onClose: () => void;
  closeOnBackdrop?: boolean;
  closeOnEscape?: boolean;
  size?: 'sm' | 'md' | 'lg' | 'xl' | 'full';
  animation?: 'fade' | 'slide' | 'scale';
}
```

Applications - Complete Page Analysis

FE-App (B2B Application) - Detailed Page Breakdown

1. Investment Funds Page (`/investment_funds`)

Location: `apps/fe-app/app/(dashboard)/(nofooter)/investment_funds/page.tsx`

This is the most complex page in the application, serving as the main dashboard for investment fund management.

Implementation Architecture

State Management:

```
const [activeIndexCategoryTab, setActiveIndexCategoryTab] =
        useState(1);
const [isShowDatePicker, setIsShowDatePicker] = useState(false);
const [customColumnDate, setCustomColumnDate] = useState({
  start: '', end: '', groupId: ''
});
const [isFilterModalOpen, setIsFilterModalOpen] = useState(false);
const [isSettingModalOpen, setIsSettingModalOpen] = useState(false);
const [localColumns, setLocalColumns] =
        useState<FundTableTabColumnDto[]>([]);
```

```
const [fundSearchQuery, setFundSearchQuery] = useState<string>('');
const [pinnedList, setPinnedList] = useState<number[]>([]);
const [rowsMark, setRowsMark] = useState<{ color: string; id: number }
        []>([]);
```

Key Features

1. Advanced Table Management - **Drag & Drop Columns**: Uses `@dnd-kit/core` for column reordering - **Column Visibility**: Dynamic show/hide columns - **Sticky Columns**: Pin important columns to left/right - **Custom Date Columns**: Add date-range based columns - **Row Marking**: Color-code rows for visual organization - **Pinned Rows**: Pin important funds to top

2. Real-time Data Integration

```
const {
  data: fundsTableData,
  isLoading: isFundsTableLoading,
  error: fundsTableError,
} = useFundsServiceGetFundsTable({
  tabId: activeIndexCategoryTab,
  searchQuery: fundSearchQuery,
});
```

3. Export Functionality - Excel export with custom formatting - PDF generation - Filtered data export - Custom column selection for export

4. Advanced Filtering System - Multi-criteria filtering - Date range filters - Numerical range filters - Text search with debouncing - Filter persistence across sessions

5. Performance Optimizations - Virtual scrolling for large datasets - Smart table scroll with `useSmartTableScroll` - Debounced search queries - Optimistic updates for user interactions

Component Structure

```
// Main table with drag-and-drop support
<DndContext
  collisionDetection={closestCenter}
  modifiers={[restrictToHorizontalAxis]}
  onDragEnd={handleDragEnd}
  sensors={sensors}
>
  <SortableContext
    items={columnOrder}
    strategy={horizontalListSortingStrategy}
  >
    <GeneralTable
```

```
      data={processedData}
      columns={dynamicColumns}
      sorting={sorting}
      onSortingChange={setSorting}
      // ... other props
    />
  </SortableContext>
</DndContext>
```

API Integration

The page integrates with multiple API endpoints: - `getFundsTable` - Main table data - `postFundsTableTabByTabColumn` - Column management - `postFundsTableTabByTabSort` - Sorting preferences - `postFundsTableTabByTabColumnsReset` - Reset column layout

2. My Fund Page (`/my-fund`)

Location: `apps/fe-app/app/(dashboard)/(withfooter)/my-fund/page.tsx`

This page provides detailed analysis of a specific investment fund with comprehensive data visualization.

Server-Side Data Fetching

```
const [
  { fundBasicInfo, fundSummaryBasicInfo, fundSummaryCaseByCase,
        fundVideoPlaylist },
  { returnComparison, returnRank, returnTrend, riskReturnAnalysis,
        seasonalityEffectAnalysis }
] = await Promise.all([
  FundsService.getFundsStockByFundIdSummary({ fundId }),
  FundsService.getFundsStockByFundIdReturnAnalysis({ fundId })
]);
```

Key Components

1. Fund Header Section - Fund logo with dynamic theming - Fund name and basic information - Watch list toggle functionality - Breadcrumb navigation

2. Tabbed Interface

```
<FundTabs
  summary={{
    points: fundSummaryCaseByCase!.navHistory.map((item) => ({
      date: item.jdt,
      value: item.revokeNavRials,
    })),
    defaultQuantity: fundSummaryCaseByCase!.navEndOfPeriodRials,
```

```
    defaultValueChange: fundSummaryCaseByCase!.returnEndOfPeriodRials,
    defaultPercentageChange:
        fundSummaryCaseByCase!.returnEndOfPeriodPercent,
    fundSummaryBasicInfo: fundSummaryBasicInfo!,
    fundSummaryCaseByCase: fundSummaryCaseByCase!,
    fundVideoPlaylist: fundVideoPlaylist!,
  }}
  returnAnalysis={{
    returnComparison, returnRank, returnTrend,
    riskReturnAnalysis, seasonalityEffectAnalysis,
  }}
/>
```

3. Data Visualization Components - **LineChart**: NAV history visualization - **BubbleChart**: Risk-return analysis - **ChangeComparisonFunds**: Comparative analysis - **VideoPlayer Integration**: Educational content

Tab Structure

Summary Tab (`Summary.tsx`): - NAV history chart with interactive tooltips - Key performance metrics - Fund composition breakdown - Recent performance indicators

Return Analysis Tab (`Return.tsx`): - Comparative return analysis - Benchmark comparisons - Historical performance trends - Risk-adjusted returns

Risk Assessment Tab (`Risk.tsx`): - Risk metrics visualization - Volatility analysis - Drawdown analysis - Risk-return scatter plots

3. Profile Page (`/profile`)

Location: `apps/fe-app/app/(dashboard)/(withfooter)/profile/page.tsx`

User profile management with comprehensive settings and preferences.

Features

- Personal information management
- Investment preferences
- Notification settings
- Security settings
- Account verification status

4. Reports Pages (`/reports`, `/report`)

Location: `apps/fe-app/app/(dashboard)/(withfooter)/reports/` and `/report/`

Comprehensive reporting system for investment analysis.

Report Types

- Portfolio performance reports
- Fund analysis reports
- Risk assessment reports
- Comparative analysis reports
- Custom date range reports

Export Capabilities

- PDF generation with custom templates
- Excel export with multiple sheets
- Email delivery system
- Scheduled report generation

B2C-App (Consumer Application) - Page Analysis

1. My Portfolio Page (`/my-portfolio`)

Location: `apps/b2c-app/app/(withfooter)/my-portfolio/page.tsx`

Simplified portfolio overview for retail investors.

Implementation

```
const MyPortfolio = () => {
  return (
    <div className="flex w-full justify-center">
      <MyPortfolioPage
        points={[
          { date: '2025-06-28', value: 123456789 },
          { date: '2025-06-29', value: 234567891 },
          // ... more data points
        ]}
        defaultQuantity={560000000}
        defaultValueChange={100000}
        defaultPercentageChange={5.3}
      />
    </div>
  );
};
```

Features

- Simplified portfolio visualization
- Basic performance metrics

- Mobile-optimized interface
- Educational tooltips

2. Profile Management (`/(profile)`)

Location: `apps/b2c-app/app/(withfooter)/(profile)/`

Consumer-focused profile management with simplified interface.

Features

- Basic personal information
- Investment goals setting
- Risk tolerance assessment
- Notification preferences

3. FAQ Page (`/faqs`)

Location: `apps/b2c-app/app/(withfooter)/faqs/`

Comprehensive FAQ system for consumer education.

Features

- Searchable FAQ database
- Categorized questions
- Interactive help system
- Contact form integration

4. Authentication Pages (`/(auth)`)

Location: `apps/b2c-app/app/(auth)/`

Simplified authentication flow for consumers.

Features

- Social login integration
- Email verification
- Password recovery
- Two-factor authentication setup

Development Setup

Prerequisites

- **Node.js**: Version 18+ (LTS recommended)
- **pnpm**: Version 8+ (package manager)
- **Nx CLI**: Global installation recommended

Installation Steps

1. **Install Global Dependencies**:

   ```
   npm install -g nx pnpm
   ```

2. **Clone and Setup**:

   ```
   git clone <repository-url>
   cd aress-frontend
   pnpm install
   ```

3. **Environment Configuration**:

   ```
   # Copy environment templates
   cp apps/fe-app/.env.local.example apps/fe-app/.env.local
   # Configure your environment variables
   ```

Development Commands

Application Development

```
# Run main B2B application
nx dev fe-app

# Run B2C application
nx dev b2c-app

# Run both applications
nx run-many --target=dev --projects=fe-app,b2c-app
```

Design System Development

```
# Run Storybook for component development
nx storybook design-system
```

```
# Build design system
nx build design-system
```

API Integration

OpenAPI/Swagger Integration

The project uses a sophisticated API integration strategy with automatic code generation:

Workflow

1. **API Specification**: OpenAPI/Swagger definitions
2. **Code Generation**: Automatic TypeScript client generation
3. **React Query Integration**: Generated hooks for data fetching
4. **Type Safety**: Full TypeScript support

Commands

```
# Merge multiple Swagger files
pnpm merge:swagger

# Generate TypeScript API client
pnpm generate:sdk

# Generate React Query hooks
pnpm generate:query

# Complete API update workflow
pnpm output:swagger
```

Generated Structure

```
libs/openapi/src/
├── queries/                    # React Query hooks
├── requests/                   # API request functions
└── types/                      # TypeScript interfaces
```

Testing Strategy

Testing Pyramid

Unit Tests (Vitest)

- **Component Testing**: Individual component behavior
- **Utility Testing**: Helper function validation
- **Hook Testing**: Custom hook functionality

Integration Tests (Jest)

- **API Integration**: Service layer testing
- **Component Integration**: Multi-component workflows
- **State Management**: Complex state scenarios

E2E Tests (Playwright)

- **User Journeys**: Critical application flows
- **Cross-browser Testing**: Chrome, Firefox, Safari
- **Mobile Testing**: Responsive behavior validation

Deployment

Build Process

Production Builds

```
# Build all applications
nx build

# Build specific application
nx build fe-app
nx build b2c-app

# Build design system
nx build design-system
```

Docker Deployment

```
# Multi-stage Docker builds
docker build -f Dockerfile.fe-app -t aress-fe-app:latest .
```

```
docker build -f Dockerfile.b2c-app -t aress-b2c-app:latest .
```

Deployment Targets

Vercel Integration

- **Automatic Deployments**: Git-based deployments
- **Preview Deployments**: PR-based previews
- **Environment Variables**: Secure configuration management

Container Orchestration

- **Docker Compose**: Local development
- **Kubernetes**: Production orchestration
- **Load Balancing**: High availability setup

Best Practices & Guidelines

Component Development

1. **Single Responsibility**: Each component has one clear purpose
2. **Prop Interface**: Well-defined TypeScript interfaces
3. **Default Props**: Sensible defaults for optional props
4. **Error Boundaries**: Graceful error handling
5. **Accessibility**: WCAG 2.1 compliance

Performance Optimization

1. **Code Splitting**: Route-based and component-based splitting
2. **Lazy Loading**: Dynamic imports for heavy components
3. **Memoization**: React.memo for expensive components
4. **Bundle Analysis**: Regular bundle size monitoring

Security Considerations

1. **Input Validation**: Client and server-side validation
2. **XSS Prevention**: Proper data sanitization

3. **CSRF Protection**: Token-based protection
4. **Dependency Scanning**: Regular security audits

Troubleshooting

Common Issues

Build Errors

- **TypeScript Errors**: Check type definitions and imports
- **Missing Dependencies**: Verify package.json and node_modules
- **Path Resolution**: Check tsconfig.json path mappings

Development Issues

- **Hot Reload**: Restart development server
- **Port Conflicts**: Check for running processes
- **Cache Issues**: Clear Nx cache with `nx reset`

This comprehensive documentation provides detailed implementation insights for developers working with the Aress Frontend Project. For the most current information, please refer to the project repository and Storybook documentation.